

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: ПОИСК ПОДСТРОКИ В СТРОКЕ

Студентка гр. 4343

Геращенко М.Д.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

1. Цель работы

Цель лабораторной работы — изучить алгоритм Кнута-Морриса-Пратта (КМП), реализовать его для поиска всех вхождений шаблона в текст, а также применить этот алгоритм для проверки циклического сдвига двух строк.

2. Постановка задачи

В лабораторной работе рассматривались следующие задачи:

- подсчёт количества сравнений в наивном алгоритме поиска подстроки;
- вычисление значений префикс-функции для заданного шаблона;
- реализация алгоритма КМП для поиска всех вхождений шаблона P в текст T ;
- определение, является ли строка A циклическим сдвигом строки B .

Для основной задачи ограничения на длины строк достигают нескольких миллионов символов, поэтому важно не только линейное время работы, но и экономное использование памяти.

3. Теоретические сведения

3.1. Наивный алгоритм поиска подстроки

Наивный алгоритм проверяет шаблон во всех возможных позициях текста. Для каждого сдвига символы шаблона сравниваются с соответствующими символами текста до первого несовпадения или до полного совпадения.

Если длина текста равна n , а длина шаблона равна m , то количество возможных сдвигов равно $n - m + 1$. В худшем случае на каждом сдвиге выполняется до m сравнений, поэтому временная сложность наивного алгоритма равна $O(nm)$.

Сдвиг	Сравнения для $T = abacabca$, $P = aba$	Количество
0	$a=a$, $b=b$, $a=a$ — найдено	3
1	$b \neq a$	1
2	$a=a$, $c \neq b$	2
3	$c \neq a$	1

4	$a=a, b=b, c \neq a$	3
5	$b \neq a$	1

Общее число сравнений в данном примере: $3 + 1 + 2 + 1 + 3 + 1 = 11$.

3.2. Префикс-функция

Префикс-функция $pi[i]$ для строки s — это длина наибольшего собственного префикса подстроки $s[0:i+1]$, который одновременно является её суффиксом. Собственный префикс — это префикс, не равный всей строке.

Например, для строки $P = aba$ значения префикс-функции равны 0 0 1, потому что для подстроки aba наибольший собственный префикс, совпадающий с суффиксом, — это строка a .

Для шаблона $P = efefeftef$ значения префикс-функции равны: 0 0 1 2 3 4 0 1 2.

Префикс-функция используется в КМП для выбора позиции, к которой нужно откатить указатель шаблона при несовпадении символов. Это позволяет не возвращаться назад по тексту.

3.3. Алгоритм Кнута-Морриса-Пратта

Алгоритм КМП выполняет поиск шаблона P в тексте T за линейное время. Сначала для шаблона строится префикс-функция. Затем алгоритм последовательно просматривает текст, поддерживая переменную j — количество символов шаблона, уже совпавших с текущим фрагментом текста.

Если очередной символ текста совпадает с символом шаблона $P[j]$, значение j увеличивается. Если происходит несовпадение, j откатывается с помощью значения $pi[j - 1]$. При этом указатель по тексту не возвращается назад.

Сложность алгоритма КМП: время $O(|P| + |T|)$, память $O(|P|)$.

4. Описание реализации

4.1. Функция вычисления префикс-функции

Функция `prefix_function(p)` строит массив pi для строки p . В реализации используется `array('I')`, так как обычный список Python с целыми числами занимает значительно больше памяти при больших ограничениях.

Переменная i задаёт текущую позицию строки, для которой вычисляется значение $pi[i]$. Переменная j хранит длину текущего совпавшего префикса. Если символ $p[i]$ не совпадает с $p[j]$, значение j уменьшается по уже вычисленной префикс-функции. Если символы совпадают, совпадение продлевается на один символ.

4.2. Поиск всех вхождений шаблона в текст

Для поиска всех вхождений шаблона P в текст T используется стандартный проход КМП. После нахождения полного совпадения индекс начала равен $i - \text{len}(P) + 1$. Затем j откатывается в $pi[j - 1]$, чтобы находить в том числе перекрывающиеся вхождения.

4.3. Проверка циклического сдвига

Строка A является циклическим сдвигом строки B тогда и только тогда, когда строки имеют одинаковую длину и строка B входит в строку $A + A$. Например, если $A = \text{defabc}$, то $A + A = \text{defabcdefabc}$, и строка $B = \text{abcdef}$ начинается в ней с индекса 3.

Однако при ограничениях до 5 000 000 символов нельзя без необходимости создавать строку $A + A$ длиной до 10 000 000 символов. Поэтому в программе используется виртуальный проход по $A + A$: символ с позиции i берётся как $A[i \% n]$.

Цикл выполняется по индексам от 0 до $2n - 2$. Этого достаточно для проверки всех возможных сдвигов, начинающихся в позициях от 0 до $n - 1$. Позиция n соответствует повтору начала строки во второй копии и не нужна.

5. Тестирование

№	Входные данные	Ожидаемый результат	Комментарий
1	defabc abcdef	3	В начинается в $A + A$ с индекса 3
2	abcdef abcdef	0	строки уже совпадают
3	aaaa aaaa	0	несколько сдвигов, нужен первый
4	abc abcd	-1	разные длины

5	abac caba	-1	циклического сдвига нет
6	bcda abcd	3	В входит в A + A с позиции 3

Тест из условия: для $A = \text{defabc}$ и $B = \text{abcdef}$ программа выводит 3.

6. Анализ сложности

Пусть $n = |A| = |B|$. Построение префикс-функции для строки B выполняется за $O(n)$. Поиск строки B в виртуальной строке $A + A$ также выполняется за $O(n)$, так как указатель по тексту только увеличивается, а откаты указателя по шаблону суммарно линейны.

Итоговая временная сложность решения: $O(n)$.

Итоговая память: $O(n)$, так как хранится префикс-функция для строки B . Для уменьшения расхода памяти используется `array('I')` вместо обычного списка Python.

7. Вывод

В ходе лабораторной работы был изучен алгоритм Кнута-Морриса-Пратта. Было показано, что использование префикс-функции позволяет выполнять поиск подстроки за линейное время, избегая повторных сравнений символов текста. Также алгоритм был применён к задаче проверки циклического сдвига строк.

Для прохождения больших ограничений программа не создаёт строку $A + A$ явно, а проходит по ней виртуально с помощью выражения $A[i \% n]$.

Дополнительно префикс-функция хранится в компактном массиве `array('I')`, что уменьшает потребление памяти.